

Overall Note

Mistral was able to one-shot everything on the homework except the learning rate and weight scale in the last question (which it didn't get even after further prompting with a hint on the two being unequal). I didn't need to provide any text to it from the prompts either, just the cells containing to-do code components.

Chat Log with comments

Prompt: Implement these functions

```
import numpy as np

def affine_forward(x, w, b): """ Computes the forward pass for an affine
(fully-connected) layer. The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
examples, where each example x[i] has shape (d_1, ..., d_k). We will reshape each input into
a vector of dimension D = d_1 * ... * d_k, and then transform it to an output vector of
dimension M. Inputs: - x: A numpy array containing input data, of shape (N, d_1, ..., d_k) -
w: A numpy array of weights, of shape (D, M) - b: A numpy array of biases, of shape (M,)
Returns a tuple of: - out: output, of shape (N, M) - cache: (x, w, b) """ out = None
#####
# # TODO: Implement the affine forward pass. Store the result in out. You # # will need to
reshape the input into rows. #
#####
#
#####
# # END OF YOUR CODE #
#####
# cache = (x, w, b) return out, cache
def affine_backward(dout, cache): """ Computes the
backward pass for an affine layer. Inputs: - dout: Upstream derivative, of shape (N, M) -
cache: Tuple of: - x: Input data, of shape (N, d_1, ... d_k) - w: Weights, of shape (D, M)
Returns a tuple of: - dx: Gradient with respect to x, of shape (N, d1, ..., d_k) - dw: Gradient
with respect to w, of shape (D, M) - db: Gradient with respect to b, of shape (M,) """ x, w, b
= cache dx, dw, db = None, None, None
#####
# # TODO: Implement the affine backward pass. #
#####
#
#####
# # END OF YOUR CODE #
#####
# return dx, dw, db
def relu_forward(x): """ Computes the forward pass for a layer of
rectified linear units (ReLUs). Input: - x: Inputs, of any shape Returns a tuple of: - out:
Output, of the same shape as x - cache: x """ out = None
#####
# # TODO: Implement the ReLU forward pass. #
#####
```

```

#
#####
# #                END OF YOUR CODE                #
#####
# cache = x return out, cachedef relu_backward(dout, cache): """ Computes the
backward pass for a layer of rectified linear units (ReLU). Input: - dout: Upstream
derivatives, of any shape - cache: Input x, of same shape as dout Returns: - dx: Gradient
with respect to x """ dx, x = None, cache
#####
# # TODO: Implement the ReLU backward pass. #
#####
#
#####
# #                END OF YOUR CODE                #
#####
# return dxdef svm_loss(x, y): """ Computes the loss and gradient using for multiclass
SVM classification. Inputs: - x: Input data, of shape (N, C) where x[i, j] is the score for the jth
class for the ith input. - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
0 <= y[i] < C Returns a tuple of: - loss: Scalar giving the loss - dx: Gradient of the loss with
respect to x """ N = x.shape[0] correct_class_scores = x[np.arange(N), y] margins =
np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0) margins[np.arange(N), y] = 0
loss = np.sum(margins) / N num_pos = np.sum(margins > 0, axis=1) dx = np.zeros_like(x)
dx[margins > 0] = 1 dx[np.arange(N), y] -= num_pos dx /= N return loss, dxdef
softmax_loss(x, y): """ Computes the loss and gradient for softmax classification. Inputs:
- x: Input data, of shape (N, C) where x[i, j] is the score for the jth class for the ith input. - y:
Vector of labels, of shape (N,) where y[i] is the label for x[i] and 0 <= y[i] < C Returns a
tuple of: - loss: Scalar giving the loss - dx: Gradient of the loss with respect to x """ probs
= np.exp(x - np.max(x, axis=1, keepdims=True)) probs /= np.sum(probs, axis=1,
keepdims=True) N = x.shape[0] loss = -np.sum(np.log(probs[np.arange(N), y])) / N dx =
probs.copy() dx[np.arange(N), y] -= 1 dx /= N return loss, dx

```

Response:

Here are the implementations for the requested functions:

python

Copy

```
import numpy as np
```

```
def affine_forward(x, w, b):
```

```
    """
```

```
    Computes the forward pass for an affine (fully-connected) layer.
```

```
    """
```

```

    N = x.shape[0]
    D = np.prod(x.shape[1:])
    x_reshaped = x.reshape(N, D)
    out = np.dot(x_reshaped, w) + b
    cache = (x, w, b)
    return out, cache

def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.
    """
    x, w, b = cache
    N = x.shape[0]
    D = np.prod(x.shape[1:])
    x_reshaped = x.reshape(N, D)
    dx = np.dot(dout, w.T).reshape(x.shape)
    dw = np.dot(x_reshaped.T, dout)
    db = np.sum(dout, axis=0)
    return dx, dw, db

def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units
    (ReLU).
    """
    out = np.maximum(0, x)
    cache = x
    return out, cache

def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units
    (ReLU).
    """
    x = cache
    dx = dout * (x > 0)
    return dx

```

```

def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM
    classification.
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] +
1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx

def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx

```

Key Points:

- **affine_forward**: Reshapes the input and computes the affine transformation.
- **affine_backward**: Computes gradients for the input, weights, and biases.
- **relu_forward**: Applies the ReLU activation function.
- **relu_backward**: Computes the gradient for ReLU.
- **svm_loss**: Computes the multiclass SVM loss and its gradient.
- **softmax_loss**: Computes the softmax loss and its gradient.

Here Mistral is slightly different from my solution in that it uses `np.dot` instead of `@` for multiplication, but this is equivalent.

Prompt: complete all todo sections

```
model = TwoLayerNet()
```

```
solver = None
```

```
#####
```

```
##
```

```
# TODO: Use a Solver instance to train a TwoLayerNet that achieves at least #
```

```
# 50% accuracy on the validation set. #
```

```
#####
```

```
##
```

```
#####
```

```
##
```

```
# END OF YOUR CODE #
```

```
#####
```

```
##
```

```
# Run this cell to visualize training loss and train / val accuracy, and save the log file of the
```

```
# experiment for submission.
```

```
plt.subplot(2, 1, 1)
```

```
plt.title('Training loss')
```

```
plt.plot(solver.loss_history, 'o')
```

```
plt.xlabel('Iteration')
```

```
plt.subplot(2, 1, 2)
```

```
plt.title('Accuracy')
```

```
plt.plot(solver.train_acc_history, '-o', label='train')
```

```
plt.plot(solver.val_acc_history, '-o', label='val')
```

```
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
```

```
plt.xlabel('Epoch')
```

```
plt.legend(loc='lower right')
```

```
plt.gcf().set_size_inches(15, 12)
```

```
plt.show()
```

```
# TODO: Use a three-layer Net to overfit 50 training examples.
```

```
num_train = 50
```

```
small_data = {
```

```
'X_train': data['X_train'][:num_train],
```

```
'y_train': data['y_train'][:num_train],
```

```
'X_val': data['X_val'],
```

```
'y_val': data['y_val'],
```

```
}
```

```
#####
```

```
#####
```

```
#####
```

```

# END OF YOUR CODE #
#####
model = FullyConnectedNet([100, 100],
weight_scale=weight_scale, dtype=np.float64)
solver = Solver(model, small_data,
print_every=10, num_epochs=20, batch_size=25,
update_rule='sgd',
optim_config={
'learning_rate': learning_rate,
}
)
solver.train()
plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
## TODO: Use a five-layer Net to overfit 50 training examples.
num_train = 50
small_data = {
'X_train': data['X_train'][:num_train],
'y_train': data['y_train'][:num_train],
'X_val': data['X_val'],
'y_val': data['y_val'],
}
#####
# TODO: Tune these parameters to get 100% train accuracy within 20 epochs. #
#####
#####
# END OF YOUR CODE #
#####
model = FullyConnectedNet([100, 100, 100, 100],
weight_scale=weight_scale, dtype=np.float64)
solver = Solver(model, small_data,
print_every=10, num_epochs=20, batch_size=25,
update_rule='sgd',
optim_config={
'learning_rate': learning_rate,
}
)
solver.train()
plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')

```

```
plt.ylabel('Training loss')
plt.show()
```

Here's how you can complete the TODOs in your code. I'll provide the missing sections for training a `TwoLayerNet`, overfitting a three-layer net, and overfitting a five-layer net on 50 training examples.

1. Training a `TwoLayerNet` to achieve at least 50% validation accuracy

python

Copy

```
model = TwoLayerNet()
solver = Solver(model, data,
                 print_every=100, num_epochs=10, batch_size=100,
                 update_rule='sgd',
                 optim_config={
                     'learning_rate': 1e-3,
                 })
solver.train()
```

Explanation:

- Use the `Solver` class to train the `TwoLayerNet`.
 - Adjust `learning_rate`, `batch_size`, and `num_epochs` as needed to achieve at least 50% validation accuracy.
-

2. Overfitting 50 training examples with a three-layer net

python

Copy

```
num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}
```

```

weight_scale = 1e-2
learning_rate = 1e-2

model = FullyConnectedNet([100, 100],
                           weight_scale=weight_scale, dtype=np.float64)
solver = Solver(model, small_data,
                print_every=10, num_epochs=20, batch_size=25,
                update_rule='sgd',
                optim_config={
                    'learning_rate': learning_rate,
                })
solver.train()
plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()

```

Explanation:

- Use a small subset of the training data (`num_train = 50`).
- Tune `weight_scale` and `learning_rate` to overfit the small dataset.

3. Overfitting 50 training examples with a five-layer net

python

Copy

```

num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}
weight_scale = 1e-2
learning_rate = 1e-2

model = FullyConnectedNet([100, 100, 100, 100],

```



```

        weight_scale=weight_scale, dtype=np.float64)
solver = Solver(model, small_data,
                print_every=10, num_epochs=20, batch_size=25,
                update_rule='sgd',
                optim_config={
                    'learning_rate': learning_rate,
                })
solver.train()
plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()

```

Explanation:

- Use a five-layer net with hidden layer sizes of 100.
 - Tune `weight_scale` and `learning_rate` to achieve 100% training accuracy within 20 epochs.
-

Notes:

- For overfitting, you may need to adjust `weight_scale` and `learning_rate` to ensure the model fits the small dataset perfectly.
- The `Solver` class is assumed to be defined elsewhere in your codebase, with methods like `train()` and attributes like `loss_history`, `train_acc_history`, and `val_acc_history`.

Seeing that it guessed a `learning_rate` and `weight_scale` of $1e-2$ for the last question, I had to guide it towards the correct rates:

Prompt: if you had to guess, which of the two should be less than the other in the last question -- the learning rate or the weight scale?

Response: In the context of overfitting a small dataset (like 50 examples) with a deep network (e.g., five layers), the learning rate should generally be larger than the weight scale.

Why?

Weight scale (`weight_scale`) controls the initial magnitude of the weights. For deep networks, it's common to use a small weight scale (e.g., $1e-2$ or smaller) to avoid exploding gradients and ensure stable training.

Learning rate (`learning_rate`) controls how much the weights are updated in response to the estimated error. To overfit a small dataset, you typically need a larger learning rate (e.g., $1e-2$ or higher) to allow the model to quickly adjust its weights and fit the training data perfectly.

Rule of Thumb for Overfitting:

Learning rate > Weight scale

(e.g., `learning_rate` = $1e-2$, `weight_scale` = $1e-3$ or $1e-2$)

This ensures the model can make large updates to its weights and fit the small dataset, even if it means sacrificing generalization. If the learning rate is too small, the model may not converge quickly enough to overfit. If the weight scale is too large, the initial gradients may explode, making training unstable.

This is not the right answer, in fact it thought the weight scale should be less than the learning rate when it should be more for overfitting.